

**PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240159 123240174
NAMA	:	Dimas Hafid Fathoni Jauza Ilham Mahardika Putra
PLUG	:	IF-G
NAMA ASISTEN	:	Farhannivta Ramadhana Muhammad Ruhul Jadid

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Dimas Hafid Fathoni

123240159

Jauza Ilham Mahardika Putra

123240174

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Asisten Praktikum

Farhannivta Ramadhana
NIM. 123230139

Muhammad Ruhul Jadid
NIM. 123230046

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul SPK Pemilihan GPU Card Terbaik Berdasarkan Kategori Tertentu. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 28 Mei 2026

Penyusun 1,

Penyusun 2,

Dimas Hafid Fathoni

NIM. 1232240159

Jauza Ilham Mahardhika Putra

NIM. 1232240174

DAFTAR ISI

HALAMAN PENGESAHAN	i
KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang Masalah	1
1.2 Tujuan Proyek Akhir	1
1.3 Manfaat Proyek Akhir	2
BAB II PEMBAHASAN	3
2.1 Dasar Teori	3
2.2 Skenario Kasus & Dataset	3
2.3 Pemodelan Sistem Pendukung Keputusan	4
2.4 Perhitungan & Algoritma	4
2.5 Implementasi & Tampilan Sistem	5
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS	42
3.1 Jadwal Pengerjaan	42
3.2 Pembagian Tugas	42
BAB IV KESIMPULAN DAN SARAN	43
4.2 Kesimpulan	43
4.3 Saran	43
DAFTAR PUSTAKA	44

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Perkembangan teknologi komputasi, khususnya pada sektor pengolahan grafis, melaju dengan sangat pesat. *Graphics Processing Unit* (GPU) kini menjadi salah satu komponen krusial tidak hanya bagi para *gamer*, tetapi juga bagi profesional di bidang *content creation*, animasi 3D, hingga pengembangan kecerdasan buatan (AI). Namun, banyaknya variasi GPU yang tersedia di pasar dengan spesifikasi yang sangat beragam—seperti kapasitas memori, kecepatan *clock*, jumlah *core*, dan hasil *benchmark*—seringkali membuat konsumen kebingungan. Terlebih lagi, fluktuasi harga GPU di pasaran membuat proses pengambilan keputusan untuk membeli GPU dengan rasio harga-performa (*price-to-performance*) terbaik menjadi semakin kompleks.

Oleh karena itu, diperlukan sebuah Sistem Pendukung Keputusan (SPK) yang mampu memproses berbagai spesifikasi teknis dan harga menjadi sebuah rekomendasi yang objektif. Proyek ini mengusulkan penggunaan metode *Simple Additive Weighting* (SAW) untuk mengevaluasi dan meranking berbagai alternatif GPU berdasarkan kriteria-kriteria yang disesuaikan dengan preferensi pengguna (seperti kebutuhan *gaming* atau *content creation*) serta batasan anggaran (*budget*).

1.2 Tujuan Proyek Akhir

Tujuan dari pembuatan proyek akhir ini adalah:

1. Membangun sebuah Sistem Pendukung Keputusan (SPK) berbasis web menggunakan antarmuka *Streamlit* untuk membantu pengguna memilih GPU terbaik.
2. Mengimplementasikan algoritma *Simple Additive Weighting* (SAW) untuk meranking alternatif GPU berdasarkan kriteria spesifikasi dan harga.
3. Mengintegrasikan berbagai sumber data (*web scraping* spesifikasi, harga, dan *benchmark*) ke dalam satu *dataset* terpadu untuk evaluasi sistem.

1.3 Manfaat Proyek Akhir

Adapun manfaat dari sistem yang dibangun antara lain:

1. Memberikan kemudahan bagi konsumen awam maupun antusias PC dalam menentukan pilihan GPU yang paling optimal sesuai dengan anggaran maksimal yang mereka miliki.
2. Menghemat waktu riset pengguna dengan menyediakan perbandingan matriks spesifikasi (seperti memori, *benchmark*, dan tahun rilis) secara otomatis di dalam satu platform.
3. Memberikan rekomendasi yang fleksibel, di mana pengguna dapat menyesuaikan bobot penilaian berdasarkan kebutuhan spesifik mereka (*gaming*, *price to performance*, atau kreasi konten).

BAB II PEMBAHASAN

2.1 Dasar Teori

Metode *Simple Additive Weighting* (SAW) atau yang sering dikenal dengan istilah metode penjumlahan terbobot adalah salah satu algoritma paling umum dan populer dalam Sistem Pendukung Keputusan. Konsep dasar metode SAW adalah mencari penjumlahan terbobot dari *rating* kinerja pada setiap alternatif di semua atribut.

Metode SAW membutuhkan proses normalisasi matriks keputusan (X) ke suatu skala yang dapat diperbandingkan dengan semua *rating* alternatif yang ada.

Formula normalisasi metode SAW adalah sebagai berikut:

$$r_{ij} = \begin{cases} \frac{x_{ij}}{\max_i x_{ij}} & \text{jika } j \text{ adalah atribut benefit (keuntungan)} \\ \frac{\min_i x_{ij}}{x_{ij}} & \text{jika } j \text{ adalah atribut cost (biaya)} \end{cases}$$

Setelah didapatkan matriks ternormalisasi (R), nilai akhir preferensi (V) untuk setiap alternatif dihitung dengan mengalikan matriks normalisasi dengan bobot kriteria (W), sesuai formula berikut:

$$V_i = \sum_{j=1}^n w_j r_{ij}$$

Alternatif dengan nilai V terbesar akan dipilih sebagai alternatif terbaik.

2.2 Skenario Kasus & Dataset

Pengambil keputusan dalam sistem ini adalah calon pembeli PC atau GPU (gamer atau profesional). Pengguna seringkali memiliki batas budget tertentu (misalnya maksimal Rp 15.000.000) dan bingung menentukan GPU mana yang memberikan performa tertinggi. Tujuan akhirnya adalah sistem menampilkan perankingan Top GPU yang harganya tidak melebihi budget pengguna.

Dataset: Sistem ini menggunakan file dataset.csv yang merupakan hasil penggabungan (merge) dari beberapa sumber data:

- `techpowerup_gpus_with_images.csv`: Menyediakan spesifikasi mentah GPU seperti nama, arsitektur, ukuran memori, GPU clock, memory clock, dan tahun rilis.
- `all_scraped_gpu_prices.csv`: Menyediakan data rata-rata harga GPU dalam mata uang Rupiah (IDR).
- `GPU_UserBenchmarks.csv`: Menyediakan skor benchmark dari situs UserBenchmark untuk menilai performa mentah secara objektif.

2.3 Pemodelan Sistem Pendukung Keputusan

Sistem dibangun menggunakan metode SAW dengan spesifikasi pemodelan sebagai berikut:

- **Alternatif**: Berbagai tipe GPU dari *brand* Nvidia, AMD, dan Intel yang terdapat dalam *database* (misal: GeForce RTX 5090, Radeon RX 9070 XT, dll). Hanya alternatif yang harganya di bawah *Max Budget* yang akan dimasukkan ke dalam perhitungan.
- **Kriteria dan Atribut**: Sistem menggunakan 7 kriteria penilaian:
 1. **Harga (Price)** – Atribut: *Cost*
 2. **Memory GB** – Atribut: *Benefit*
 3. **GPU Clock** – Atribut: *Benefit*
 4. **Memory Clock** – Atribut: *Benefit*
 5. **Cores** – Atribut: *Benefit*
 6. **Tahun Rilis (Release Year)** – Atribut: *Benefit*
 7. **Benchmark Score** – Atribut: *Benefit*
- **Skala Penilaian (Bobot)**: Nilai bobot (W) bersifat dinamis dan dapat diatur oleh pengguna (*user-defined*) melalui *slider* rentang **0.01 - 1.00** atau dipilih melalui *template* (Gaming, Price to Performance, Content Creation). Total seluruh bobot nantinya akan dinormalisasi ($sum = 1$).

2.4 Perhitungan & Algoritma

Langkah-langkah algoritma SAW yang diimplementasikan pada sistem adalah sebagai berikut:

1. **Validasi Budget**: Sistem akan menyaring (filter) seluruh baris pada dataset. Hanya GPU dengan harga $\leq$ *Max Budget* dan memiliki skor *benchmark* > 0 yang dievaluasi.

2. **Inisialisasi Matriks Awal (X):** Mengambil nilai asli dari dataset untuk kriteria yang aktif (Harga, Memori, *Clock*, *Core*, Tahun, *Benchmark*).
3. **Normalisasi Matriks (R):** Jika kriteria adalah Harga (*Cost*), maka nilai minimum dari seluruh alternatif dibagi dengan nilai alternatif tersebut. Jika kriteria lainnya (*Benefit*), nilai alternatif dibagi dengan nilai maksimum dari kolom kriteria tersebut.
4. **Perhitungan Preferensi (V):** Mengalikan matriks normalisasi (R) dengan vektor bobot kriteria (W) yang telah disesuaikan dengan input pengguna.
5. **Perangkingan:** Nilai V dari seluruh alternatif diurutkan secara menurun (*descending*). GPU dengan nilai V tertinggi menduduki peringkat pertama dan menjadi rekomendasi utama sistem

2.5 Implementasi & Tampilan Sistem

A. Source Code

1. gpupicker.py

File ini merupakan program inti dari sistem yang bertugas menangani dua hal: antarmuka pengguna (User Interface) dan logika perhitungan metode SPK. Dibangun menggunakan framework Streamlit, script ini akan membaca file dataset.csv dan menampilkan halaman interaktif berbasis web kepada pengguna. Pada file ini juga ditanamkan algoritma Simple Additive Weighting (SAW), di mana program akan menangkap input dari pengguna (seperti maksimal anggaran/budget dan tingkat kepentingan/bobot setiap kriteria), mem-filter GPU yang tidak sesuai budget, menghitung matriks normalisasi, hingga menghasilkan nilai preferensi akhir (V). File ini juga bertanggung jawab untuk memvisualisasikan hasil perankingan tersebut dalam bentuk daftar rekomendasi dan grafik radar (radar chart) yang mudah dipahami.

```
Font Courier New 10 spasi 1
import streamlit as st
import pandas as pd
import numpy as np
import os
import plotly.express as px
import plotly.graph_objects as go

st.set_page_config(page_title="GPU Picker", layout="wide")

# Load css
```

```

def load_css(file_name):
    if os.path.exists(file_name):
        with open(file_name) as f:
            st.markdown(f'<style>{f.read()}</style>',
unsafe_allow_html=True)
    else:
        st.warning(f"CSS file not found: {file_name}")

load_css(os.path.join("style", "base.css"))

@st.cache_data
def load_data():
    for candidate in [os.path.join("Data", "dataset.csv"),
"dataset.csv"]:
        if os.path.exists(candidate):
            return pd.read_csv(candidate)
        raise FileNotFoundError("dataset.csv not found")

def get_manufacturer(name):
    name_lower = str(name).lower()
    if 'radeon' in name_lower or 'rx ' in name_lower:
        return 'AMD'
    elif 'geforce' in name_lower or 'rtx' in name_lower or 'gtx'
in name_lower or 'gt ' in name_lower:
        return 'NVIDIA'
    elif 'arc' in name_lower or 'intel' in name_lower:
        return 'Intel'
    return 'Other'

def preprocess_data(df):
    df_eval = df.copy()
    df_eval['Memory_GB'] =
df_eval['Memory'].str.extract(r'(\d+)').astype(float)
    df_eval['GPU_Clock_MHz'] = df_eval['GPU
Clock'].str.extract(r'(\d+)').astype(float)
    df_eval['Memory_Clock_MHz'] = df_eval['Memory
Clock'].str.extract(r'(\d+)').astype(float)
    df_eval['Cores'] = df_eval['Cores / TMUs /
ROPs'].str.extract(r'^(\d+)').astype(float)
    df_eval['Price'] = df_eval['Average Price
(IDR)'].astype(float)
    df_eval['Release_Year'] = df_eval['Release
Date'].str.extract(r'(\d{4})').astype(float)
    df_eval['Benchmark_Score'] =
df_eval['Benchmark'].astype(float)

    df_eval['Manufacturer'] =
df_eval['Name'].apply(get_manufacturer)

    df_eval = df_eval.dropna(subset=['Memory_GB',
'GPU_Clock_MHz', 'Memory_Clock_MHz', 'Cores', 'Price',
'Release_Year', 'Benchmark_Score'])
    return df_eval

df_raw = load_data()
df_all = preprocess_data(df_raw)

# sidebar
with st.sidebar:

```

```

st.header("GPU Picker")
page = st.selectbox("Navigations", [
    "Page 1 - Main Page",
    "Page 2 - SAW",
    "Page 3 - Result & Recommendation"
])

st.divider()
st.markdown("### Pengaturan SPK")

# Budget untuk spk
max_budget = st.number_input(
    "Max Budget (IDR)",
    min_value=1_000_000,
    max_value=100_000_000,
    value=15_000_000,
    step=500_000,
    help="Hanya GPU di bawah budget ini yang akan dievaluasi oleh sistem rekomendasi."
)

template = st.selectbox("Pilih Template Preferensi",
["Manual", "Gaming", "Price to Performance", "Content Creation"])

all_criteria = [
    "Harga (Cost)", "Memory GB (Benefit)", "GPU Clock (Benefit)",
    "Memory Clock (Benefit)", "Cores (Benefit)", "Tahun Rilis (Benefit)", "Benchmark (Benefit)"
]

default_4_criteria = ["Harga (Cost)", "Memory GB (Benefit)", "Tahun Rilis (Benefit)", "Benchmark (Benefit)"]

# nilai pengaturan
if template == "Gaming":
    t_price, t_mem, t_gclk, t_mclk, t_core, t_year, t_bench = 0.10, 0.15, 0.10, 0.10, 0.10, 0.15, 0.30
    active_defaults = all_criteria
elif template == "Price to Performance":
    t_price, t_mem, t_gclk, t_mclk, t_core, t_year, t_bench = 0.35, 0.10, 0.0, 0.0, 0.0, 0.20, 0.35
    active_defaults = default_4_criteria
elif template == "Content Creation":
    t_price, t_mem, t_gclk, t_mclk, t_core, t_year, t_bench = 0.10, 0.25, 0.05, 0.05, 0.20, 0.15, 0.20
    active_defaults = all_criteria
else:
    t_price, t_mem, t_gclk, t_mclk, t_core, t_year, t_bench = 0.25, 0.10, 0.10, 0.10, 0.10, 0.20, 0.25
    active_defaults = default_4_criteria

st.markdown("#### Bobot Kriteria")
d_disable = template != "Manual"

active_criteria = st.multiselect(
    "Pilih Kriteria Aktif",
    options=all_criteria,

```

```

        default=active_defaults
    )

    use_price = "Harga (Cost)" in active_criteria
    use_mem = "Memory GB (Benefit)" in active_criteria
    use_gclk = "GPU Clock (Benefit)" in active_criteria
    use_mclk = "Memory Clock (Benefit)" in active_criteria
    use_core = "Cores (Benefit)" in active_criteria
    use_year = "Tahun Rilis (Benefit)" in active_criteria
    use_bench = "Benchmark (Benefit)" in active_criteria

    bar_price = bar_mem = bar_gclk = bar_mclk = bar_core = bar_year
    = bar_bench = 0.0

    if use_price:
        bar_price = st.slider("Bobot Harga", 0.01, 1.00, t_price,
0.01, disabled=d_disable)
    if use_mem:
        bar_mem = st.slider("Bobot Memory GB", 0.01, 1.00, t_mem,
0.01, disabled=d_disable)
    if use_gclk:
        bar_gclk = st.slider("Bobot GPU Clock", 0.01, 1.00,
t_gclk, 0.01, disabled=d_disable)
    if use_mclk:
        bar_mclk = st.slider("Bobot Memory Clock", 0.01, 1.00,
t_mclk, 0.01, disabled=d_disable)
    if use_core:
        bar_core = st.slider("Bobot Cores", 0.01, 1.00, t_core,
0.01, disabled=d_disable)
    if use_year:
        bar_year = st.slider("Bobot Tahun Rilis", 0.01, 1.00,
t_year, 0.01, disabled=d_disable)
    if use_bench:
        bar_bench = st.slider("Bobot Benchmark", 0.01, 1.00,
t_bench, 0.01, disabled=d_disable)

    # Menghitung bobot W
    bobot_awal = np.array([bar_price, bar_mem, bar_gclk, bar_mclk,
bar_core, bar_year, bar_bench])

    if np.sum(bobot_awal) == 0:
        st.warning("Silakan aktifkan setidaknya satu kriteria
SPK.")
        w = np.zeros_like(bobot_awal)
    else:
        w = bobot_awal / np.sum(bobot_awal)

# Page 1
if page == "Page 1 - Main Page":
    total_gpu = len(df_all)

    hero_section = f"""
<style>
@keyframes countUpAnim {{
    from {{ --num: 0; }}
    to {{ --num: {total_gpu}; }}
}}
</style>

```

```

<div class="hero-container">
  <div class="hero-col-left">
    <div class="top-left">
      <span class="hero-app-name">GPUPICKER</span>
    </div>
    <div class="bottom-left">
      <div class="brands-reveal">
        <div class="brands-rotor">
          <div class="brand-word">Nvidia</div>
          <div class="brand-word">AMD</div>
          <div class="brand-word">Intel</div>
          <div class="brand-word">Nvidia</div>
        </div>
      </div>
    </div>
  </div>
  <div class="hero-col-right">
    <div class="top-right">
      <span class="hero-number"></span>
      <span class="hero-text-large">GPU Recorded</span>
    </div>
    <div class="bottom-right">
      <span class="hero-text-large">in Database</span>
    </div>
  </div>
</div>
"""

st.markdown(hero_section, unsafe_allow_html=True)

st.title("Explore GPU Database")
st.write("Telusuri seluruh database GPU secara bebas  
menggunakan fungsionalitas pencarian di bawah ini.")

# Search Container
with st.container(border=True):
    st.markdown("#### Search")

    col1, col2, col3 = st.columns(3)
    with col1:
        search_name = st.text_input("Search GPU Name",
placeholder="example: RTX 4070")
    with col2:
        max_price = st.slider(
            "Price Range (IDR)",
            min_value=int(df_all['Price'].min()),
            max_value=int(df_all['Price'].max()),
            value=int(df_all['Price'].max()),
            step=500_000
        )
    with col3:
        max_year = st.slider(
            "Release Year Range",
            min_value=int(df_all['Release_Year'].min()),
            max_value=int(df_all['Release_Year'].max()),
            value=int(df_all['Release_Year'].max())
        )

# opsi advanced options

```

```

with st.expander("Advanced Options"):
    adv_col1, adv_col2 = st.columns([1, 2])

    with adv_col1:
        manufacturer_list = df_all['Manufacturer'].unique().tolist()
        selected_manufacturers = st.multiselect("Chose Manufacturers", manufacturer_list, default=manufacturer_list)

    with adv_col2:
        spec_col1, spec_col2, spec_col3 = st.columns(3)
        with spec_col1:
            min_mem = st.slider(
                "Minimum Memory (GB)",
                min_value=int(df_all['Memory_GB'].min()),
                max_value=int(df_all['Memory_GB'].max()),
                value=int(df_all['Memory_GB'].min())
            )
        with spec_col2:
            min_clock = st.slider(
                "Minimum GPU Clock (MHz)",
                min_value=int(df_all['GPU_Clock_MHz'].min()),
                max_value=int(df_all['GPU_Clock_MHz'].max()),
                value=int(df_all['GPU_Clock_MHz'].min())
            )
        with spec_col3:
            min_bench = st.slider(
                "Minimum Score Benchmark",
                min_value=float(df_all['Benchmark_Score'].min()),
                max_value=float(df_all['Benchmark_Score'].max()),
                value=float(df_all['Benchmark_Score'].min())
            )

    # Filter pencarian
    df_display = df_all[
        (df_all['Name'].str.contains(search_name, case=False, na=False)) &
        (df_all['Manufacturer'].isin(selected_manufacturers)) &
        (df_all['Price'] <= max_price) &
        (df_all['Release_Year'] <= max_year) &
        (df_all['Memory_GB'] >= min_mem) &
        (df_all['GPU_Clock_MHz'] >= min_clock) &
        (df_all['Benchmark_Score'] >= min_bench)
    ]

    st.markdown(f"Displaying {len(df_display)} of {len(df_all)} Total GPU inside the dataset:")
    st.data_editor(
        df_display[['Name', 'Image_URL', 'Release Date', 'Manufacturer', 'Bus', 'Memory', 'GPU Clock', 'Memory Clock', 'Cores / TMUs / ROPs', 'Benchmark', 'Average Price (IDR)']],
        column_config={
            "Image_URL": st.column_config.ImageColumn("Visual", help="Gambar GPU"),

```

```

        "Average Price (IDR)":
st.column_config.NumberColumn("Harga (IDR)", format="Rp %d"),
        "Benchmark": st.column_config.NumberColumn("Score",
format="%.1f")
    },
    use_container_width=True,
    hide_index=True,
    disabled=True
)

# page 2 dab 3 - spk saw
else:
    # Filter Dataset untuk SPK berdasar max budget dan bechmark
score
    df_eval = df_all[(df_all['Price'] <= max_budget) &
(df_all['Benchmark_Score'] > 0)].copy()

    if df_eval.empty:
        st.error(f"Tidak ada GPU valid yang ditemukan di bawah
budget Rp. {max_budget:,}. Silakan tambah Max Budget pada
Sidebar.")
        st.stop()

    # 1 - IDENTIFIKASI KRITERIA AKTIF (Filter kolom jika bobot
bernilai > 0)
    aktif_idx = [i for i, val in enumerate(bobot_awal) if val > 0]

    all_k = np.array([0, 1, 1, 1, 1, 1, 1])
    all_cols = ['Price', 'Memory_GB', 'GPU_Clock_MHz',
'Memory_Clock_MHz', 'Cores', 'Release_Year', 'Benchmark_Score']
    all_kriteria_names = ["Price (IDR)", "Memory (GB)", "GPU Clock
(MHz)", "Memory Clock (MHz)", "Cores", "Tahun Rilis", "Benchmark
Score"]

    k = all_k[aktif_idx]
    cols_aktif = [all_cols[i] for i in aktif_idx]
    kriteria_names = [all_kriteria_names[i] for i in aktif_idx]
    w_aktif = w[aktif_idx]

    # 2 - MEMBENTUK MATRIKS KEPUTUSAN (X): Hanya mengambil kolom
yang aktif
    x = df_eval[cols_aktif].values
    alternatif_names = df_eval['Name'].values

    m, n = x.shape

    # 3 - NORMALISASI MATRIKS (R)
    R = np.zeros((m, n))
    for j in range(n):
        if k[j] == 1:
            R[:, j] = x[:, j] / np.max(x[:, j])
        else:
            R[:, j] = np.min(x[:, j]) / x[:, j]

    # 4 - PERHITUNGAN PREFERENSI (V): Kalikan normalisasi dengan
bobot yg aktif
    V = np.sum(w_aktif * R, axis=1)

```

```

# 5 - PERANGKINGAN: Memasukkan nilai V ke dataframe lalu
diurutkan
df_result = df_eval.copy()
df_result['Nilai V'] = V
df_result['Ranking'] = df_result['Nilai
V'].rank(ascending=False).astype(int)
df_result =
df_result.sort_values("Ranking").reset_index(drop=True)

# Page 2 Perhitungan SAW
if page == "Page 2 - SAW":
    st.title("Metode Simple Additive Weighting (SAW)")
    st.write("Menampilkan matriks normalisasi SAW hanya untuk
GPU yang lolos validasi budget.")

    st.subheader("1. Inisialisasi Matriks Awal (x)")
    st.dataframe(pd.DataFrame(x, columns=kriteria_names,
index=alternatif_names), use_container_width=True)

    st.subheader("2. Matriks Keputusan Berdasarkan Atribut
(k)")
    st.write("Keterangan: Harga di-set 0 (Cost), Selain itu
(Tahun Rilis, Core, Benchmark) di-set 1 (Benefit)")
    df_k = pd.DataFrame([k], columns=kriteria_names,
index=["Atribut (0=Biaya, 1=Keuntungan)"])
    st.dataframe(df_k, use_container_width=True)

    st.subheader("3. Normalisasi Matriks (R)")
    st.latex(r"""
r_{ij} =
\begin{cases}
\frac{x_{ij}}{\max_i x_{ij}} & \text{jika } j \text{ \text{
adalah atribut benefit (keuntungan)} \\
\frac{\min_i x_{ij}}{x_{ij}} & \text{jika } j \text{ \text{
adalah atribut cost (biaya)} \\
\end{cases}
""")
    st.dataframe(pd.DataFrame(R, columns=kriteria_names,
index=alternatif_names).style.format("{:.4f}"),
use_container_width=True)

    st.subheader("4. Bobot (w)")
    st.dataframe(pd.DataFrame([w_aktif],
columns=kriteria_names, index=["Bobot W
(Ternormalisasi)"]).style.format("{:.4f}"),
use_container_width=True)

    st.subheader("5. Hasil Perangkingan Akhir (V)")
    st.latex(r"V_i = \sum_{j=1}^n w_j r_{ij}")
    df_v = pd.DataFrame(V, columns=["Nilai V"],
index=alternatif_names).sort_values(by="Nilai V",
ascending=False)
    st.dataframe(df_v.style.format("{:.5f}"),
use_container_width=True)

    st.markdown("""
<div class="viz-dashboard-header">
<div>

```

```

        <div                                class="viz-header-title">Data
Visualizations</div>
        <div class="viz-header-sub">Select charts below to
explore results</div>
    </div>
</div>
"""", unsafe_allow_html=True)

    plotly_layout = dict(
        paper_bgcolor='rgba(0,0,0,0)',
        plot_bgcolor='rgba(22,22,22,0.8)',
        font=dict(family="Inter, Source Sans Pro, sans-serif",
color='#c0c0c0', size=12),
        margin=dict(l=40, r=20, t=50, b=40),
        hoverlabel=dict(bgcolor='#1e1e1e',
font_color='#e0e0e0', font_size=12, bordercolor='#333'),
        legend=dict(bgcolor='rgba(0,0,0,0)',
bordercolor='#333', font=dict(size=11)),
        xaxis=dict(gridcolor='#2a2a2a',
zerolinecolor='#333'),
        yaxis=dict(gridcolor='#2a2a2a',
zerolinecolor='#333'),
    )

    MFG_COLORS = {'NVIDIA': '#76B900', 'AMD': '#ED1C24',
'Intel': '#0071C5', 'Other': '#888888'}

    st.markdown('<div                                class="viz-control-bar">',
unsafe_allow_html=True)
    ctrl_col1, ctrl_col2, ctrl_col3 = st.columns([3, 1, 1])

    available_charts = [
        "Top N GPU Ranking",
        "Criteria Weight Distribution",
        "Price vs Benchmark",
        "Radar Comparison (Top 5)",
        "Benchmark by Manufacturer",
        "Memory vs Clock Heatmap",
        "Release Year Trend",
    ]

    with ctrl_col1:
        selected_charts = st.multiselect(
            "Choose Visualizations",
            options=available_charts,
            default=["Top N GPU Ranking"],
            help="Pick which charts to display below."
        )

    with ctrl_col2:
        top_n = st.slider("GPUs to show", 5, min(30,
len(df_result)), 10, help="Controls ranking charts")

    with ctrl_col3:
        color_mode = st.selectbox("Color by", ["Manufacturer",
"Ranking Tier"], help="Color scheme for charts")

    st.markdown('</div>', unsafe_allow_html=True)

    if not selected_charts:

```

```

        st.info("Select at least one visualization from the
dropdown above.")

    def get_tier_color(rank):
        if rank <= 3: return '#FFD700'
        elif rank <= 10: return '#C0C0C0'
        elif rank <= 20: return '#CD7F32'
        return '#555555'

    def get_color(row, mode):
        if mode == "Manufacturer":
            return MFG_COLORS.get(row.get('Manufacturer',
''), '#888')
        return get_tier_color(row.get('Ranking', 99))

    def render_top_n_ranking():
        top = df_result.head(top_n).sort_values(by="Nilai V",
ascending=True)

        if color_mode == "Manufacturer":
            colors = [MFG_COLORS.get(m, '#888') for m in
top['Manufacturer']]
        else:
            colors = [get_tier_color(r) for r in
top['Ranking']]

        fig = go.Figure(go.Bar(
            x=top['Nilai V'],
            y=top['Name'],
            orientation='h',
            marker=dict(color=colors, line=dict(width=0)),
            text=[f"{v:.4f}" for v in top['Nilai V']],
            textposition='outside',
            textfont=dict(size=10, color='#aaa'),
            hovertemplate='<b>{y}</b><br>V-Score:
%{x:.5f}<br><extra></extra>',
        ))

        fig.update_layout(**plotly_layout)

        fig.update_layout(
            title=dict(text=f'Top {top_n} GPU by SAW Score',
font=dict(size=15, color='#e0e0e0')),
            xaxis_title='SAW Score (V)',
            yaxis=dict(gridcolor='rgba(0,0,0,0)'),
            height=max(350, top_n * 32),
            showlegend=False,
        )
        st.plotly_chart(fig, use_container_width=True,
key="chart_ranking")

    def render_weight_donut():
        fig = go.Figure(go.Pie(
            labels=kriteria_names,
            values=w_aktif,
            hole=0.55,
            marker=dict(
                colors=px.colors.qualitative.Set3[:len(kriteria_names)],

```

```

        line=dict(color='#1a1a1a', width=2)
    ),
    textinfo='label+percent',
    textfont=dict(size=11, color='#ccc'),
    hovertemplate='<b>{%label}</b><br>Weight:
    {%value:.4f}<br>Share: {%percent}<extra></extra>',
    ))

    fig.update_layout(**plotly_layout)

    fig.update_layout(
        title=dict(text='Criteria Weight Distribution',
font=dict(size=15, color='#e0e0e0')),
        height=400,
        showlegend=True,
        legend=dict(orientation="h", yanchor="bottom",
y=-0.15, xanchor="center", x=0.5, font=dict(size=10,
color='#aaa')),
        annotations=[dict(text='Weights', x=0.5, y=0.5,
font_size=14, font_color='#666', showarrow=False)],
    )
    st.plotly_chart(fig, use_container_width=True,
key="chart_donut")

def render_price_vs_benchmark():
    scatter_df = df_result.head(top_n).copy()

    fig = px.scatter(
        scatter_df,
        x='Price',
        y='Benchmark_Score',
        size='Memory_GB',
        color='Manufacturer',
        color_discrete_map=MFG_COLORS,
        hover_name='Name',
        hover_data={
            'Price': ':,.0f',
            'Benchmark_Score': ':.1f',
            'Memory_GB': ':.0f',
            'Ranking': True,
            'Manufacturer': False,
        },
        size_max=35,
    )

    fig.update_layout(**plotly_layout)

    fig.update_layout(
        title=dict(text='Price vs Benchmark (bubble =
Memory GB)', font=dict(size=15, color='#e0e0e0')),
        xaxis_title='Price (IDR)',
        yaxis_title='Benchmark Score',
        height=450,
    )
    fig.update_traces(marker=dict(line=dict(width=1,
color='#333'), opacity=0.85))
    st.plotly_chart(fig, use_container_width=True,
key="chart_scatter")

```

```

def render_radar():
    top5 = df_result.head(5)
    radar_cols = [c for c in cols_aktif]

    fig = go.Figure()

    color_palette = ['#76B900', '#ED1C24', '#0071C5',
'#FFD700', '#FF6B9D']

    for i, (_, row) in enumerate(top5.iterrows()):
        row_idx = df_eval.index.get_loc(row.name) if
row.name in df_eval.index else i
        if row_idx < R.shape[0]:
            r_vals = [R[row_idx, cols_aktif.index(c)] if
c in cols_aktif else 0 for c in radar_cols]
        else:
            continue
        r_vals_closed = r_vals + [r_vals[0]]
        theta_closed =
[kriteria_names[cols_aktif.index(c)] for c in radar_cols] +
[kriteria_names[cols_aktif.index(radar_cols[0])]]

        clr = color_palette[i % len(color_palette)]
        fig.add_trace(go.Scatterpolar(
            r=r_vals_closed,
            theta=theta_closed,
            name=f"#{int(row['Ranking'])} {row['Name']}",
            fill='toself',

fillcolor=f"rgba({int(clr[1:3],16)},{int(clr[3:5],16)},{int(clr[5
:7],16)},0.08)",
            line=dict(color=clr, width=2),
            hovertemplate='%{theta}:
%{r:.3f}<extra>%{fullData.name}</extra>',
        ))

    fig.update_layout(**plotly_layout)

    fig.update_layout(
        title=dict(text='Radar Comparison - Top 5 GPUs',
font=dict(size=15, color='#e0e0e0')),
        polar=dict(
            bgcolor='rgba(22,22,22,0.6)',
            radialaxis=dict(visible=True, range=[0,
1.05],
            gridcolor='#2a2a2a', linecolor='#333',
            tickfont=dict(size=9, color='#666')),
            angularaxis=dict(gridcolor='#2a2a2a',
            linecolor='#333', tickfont=dict(size=10, color='#aaa')),
        ),
        height=480,
        showlegend=True,
        legend=dict(font=dict(size=10)),
    )
    st.plotly_chart(fig, use_container_width=True,
key="chart_radar")

def render_benchmark_box():
    fig = px.box(
        df_result,

```

```

        x='Manufacturer',
        y='Benchmark_Score',
        color='Manufacturer',
        color_discrete_map=MFG_COLORS,
        points='all',
        hover_data=['Name', 'Price'],
    )

    fig.update_layout(**plotly_layout)

    fig.update_layout(
        title=dict(text='Benchmark Distribution by
Manufacturer', font=dict(size=15, color='#e0e0e0')),
        xaxis_title='',
        yaxis_title='Benchmark Score',
        height=420,
        showlegend=False,
    )
    fig.update_traces(marker=dict(size=5, opacity=0.6),
jitter=0.3)
    st.plotly_chart(fig, use_container_width=True,
key="chart_box")

def render_mem_clock_heatmap():
    fig = px.scatter(
        df_result.head(top_n),
        x='Memory_GB',
        y='GPU_Clock_MHz',
        color='Nilai V',
        color_continuous_scale=['#1a1a2e', '#16213e',
'#0f3460', '#e94560', '#FFD700'],
        hover_name='Name',
        hover_data={
            'Memory_GB': ':.0f',
            'GPU_Clock_MHz': ':.0f',
            'Nilai V': ':.4f',
            'Manufacturer': True,
        },
        size='Benchmark_Score',
        size_max=30,
    )

    fig.update_layout(**plotly_layout)

    fig.update_layout(
        title=dict(text='Memory vs GPU Clock (color = V-
Score)', font=dict(size=15, color='#e0e0e0')),
        xaxis_title='Memory (GB)',
        yaxis_title='GPU Clock (MHz)',
        height=420,
        coloraxis_colorbar=dict(
            title=dict(text='V-Score', font=dict(size=11,
color='#aaa')),
            tickfont=dict(size=10, color='#aaa')
        ),
    )
    fig.update_traces(marker=dict(line=dict(width=1,
color='#333'), opacity=0.85))

```

```

        st.plotly_chart(fig, use_container_width=True,
key="chart_heatmap")

    def render_year_trend():
        trend = df_result.groupby('Release_Year').agg(
            Avg_Benchmark=('Benchmark_Score', 'mean'),
            Count=('Name', 'count'),
            Avg_V=('Nilai V', 'mean'),
        ).reset_index()

        fig = go.Figure()
        fig.add_trace(go.Scatter(
            x=trend['Release_Year'],
            y=trend['Avg_Benchmark'],
            mode='lines+markers',
            name='Avg Benchmark',
            line=dict(color='#76B900', width=2.5,
shape='spline'),
            marker=dict(size=trend['Count'] * 2 + 4,
color='#76B900', line=dict(width=1, color='#333')),
            hovertemplate='<b>Year {x:.0f}</b><br>Avg
Benchmark: {y:.1f}<br>GPUs: {marker.size}<extra></extra>',
        ))
        fig.add_trace(go.Bar(
            x=trend['Release_Year'],
            y=trend['Count'],
            name='GPU Count',
            marker=dict(color='rgba(118,185,0,0.15)',
line=dict(width=0)),
            yaxis='y2',
            hovertemplate='Year {x:.0f}<br>Count:
{y}<extra></extra>',
        ))

        fig.update_layout(**plotly_layout)

        fig.update_layout(
            title=dict(text='Benchmark Trend by Release Year',
font=dict(size=15, color='#e0e0e0')),
            xaxis_title='Release Year',
            yaxis=dict(title='Avg Benchmark Score',
gridcolor='#2a2a2a', zerolinecolor='#333'),
            yaxis2=dict(title=dict(text='GPU Count',
font=dict(color='#555')), overlaying='y', side='right',
showgrid=False, tickfont=dict(color='#555')),
            height=400,
            legend=dict(orientation="h", yanchor="bottom",
y=1.02, xanchor="right", x=1),
        )
        st.plotly_chart(fig, use_container_width=True,
key="chart_trend")

    chart_dispatch = {
        "Top N GPU Ranking": render_top_n_ranking,
        "Criteria Weight Distribution": render_weight_donut,
        "Price vs Benchmark": render_price_vs_benchmark,
        "Radar Comparison (Top 5)": render_radar,
        "Benchmark by Manufacturer": render_benchmark_box,
        "Memory vs Clock Heatmap": render_mem_clock_heatmap,
    }

```

```

        "Release Year Trend": render_year_trend,
    }

    if selected_charts:
        for i in range(0, len(selected_charts), 2):
            pair = selected_charts[i:i+2]
            if len(pair) == 2:
                col_a, col_b = st.columns(2)
                with col_a:
                    with st.container(border=True):
                        chart_dispatch[pair[0]]()
                with col_b:
                    with st.container(border=True):
                        chart_dispatch[pair[1]]()
            else:
                with st.container(border=True):
                    chart_dispatch[pair[0]]()

    # Page 3 hasil dan rekomendasi
    elif page == "Page 3 - Result & Recommendation":
        import base64

        def load_asset_b64(paths, mime):
            for p in paths:
                if os.path.exists(p):
                    with open(p, 'rb') as f:
                        return
            f"data:{mime};base64,{base64.b64encode(f.read()).decode()}"
            return None

        gif_b64 = load_asset_b64(
            [os.path.join("Asset", "ubmlogo-60-loop.gif"),
            "ubmlogo-60-loop.gif",
            "/mnt/user-data/uploads/ubmlogo-60-loop.gif"],
            "image/gif")
        logo_b64 = load_asset_b64(
            [os.path.join("Asset", "UserBenchmarkLogo.png"),
            "UserBenchmarkLogo.png",
            "/mnt/user-data/uploads/UserBenchmarkLogo.png"],
            "image/png")

        # Benchmark csv
        @st.cache_data
        def load_userbenchmark():
            for candidate in [os.path.join("Data", "Used Data",
            "GPU_UserBenchmarks.csv"),
            os.path.join("Data",
            "GPU_UserBenchmarks.csv"), "GPU_UserBenchmarks.csv"]:
                if os.path.exists(candidate):
                    ub_path = candidate
                    break
            else:
                return pd.DataFrame()
            try:
                ub = pd.read_csv(ub_path)
                ub = ub[ub['URL'].notna() & (ub['URL'].str.strip()
                != '')]

                return ub
            except Exception:

```

```

        return pd.DataFrame()

    def find_userbenchmark_url(gpu_name, ub_df):
        if ub_df.empty:
            return None, None, None
        name_lower = str(gpu_name).lower()
        clean = name_lower.replace('geforce ',
''.replace('radeon ', '').replace('intel arc ', 'arc ')
        best_score = 0
        best_row = None
        for _, row in ub_df.iterrows():
            model = str(row.get('Model', '')).lower()
            tokens = [t for t in clean.split() if len(t) > 1]
            score = sum(1 for t in tokens if t in model)
            if score > best_score:
                best_score = score
                best_row = row
        if best_row is not None and best_score >= 2:
            return best_row.get('Rank'),
best_row.get('Benchmark'), best_row.get('URL')
        return None, None, None

    ub_df = load_userbenchmark()
    best_gpu = df_result.iloc[0]

    mfg = best_gpu['Manufacturer']
    if mfg == 'NVIDIA': mfg_color, mfg_bg = "#76B900",
"#76B90018"
    elif mfg == 'AMD': mfg_color, mfg_bg = "#ED1C24",
"#ED1C2418"
    elif mfg == 'Intel': mfg_color, mfg_bg = "#0071C5",
"#0071C518"
    else: mfg_color, mfg_bg = "#888888",
"#88888818"

    ub_rank, ub_bench, ub_url =
find_userbenchmark_url(best_gpu['Name'], ub_df)

    # inject css
    st.markdown(f"""
<style>
    :root {{
        --mfg-color: {mfg_color};
        --mfg-bg: {mfg_bg};
        --mfg-border: {mfg_color}44;
    }}
</style>
<div class="p3-root">
    <div class="p3-header">GPU Recommendation</div>
    <div class="p3-sub">GPU yang paling sesuai berdasarkan
Max Budget</div>
</div>
""", unsafe_allow_html=True)

    img_col, info_col = st.columns([1, 2], gap="large")

    with img_col:
        if pd.notna(best_gpu.get('Image_URL')):

```

```

        st.image(best_gpu['Image_URL'],
use_container_width=True)

        # score box
        st.markdown(f"""
<div class="score-box">
    <div class="score-lbl">SAW Score (Nilai V)</div>
    <div
        class="score-val">{best_gpu['Nilai
V']:.4f}</div>
</div>
""", unsafe_allow_html=True)

        # UserBenchmark container
        if ub_url:
            rank_str = f"GPU Rank
<strong>#{int(ub_rank)}</strong>" if ub_rank and not
pd.isna(ub_rank) else ""
            bench_str = f"Average Benchmark:
<strong>{float(ub_bench):.1f}%</strong>" if ub_bench and not
pd.isna(ub_bench) else ""
            stat_line = " &nbsp;&bull;&nbsp;&nbsp;"
            ".join(filter(None, [rank_str, bench_str]))
            gif_img = f'' if gif_b64 else ""

            logo_img = f'<a class="ub-site" href="{ub_url}"
target="_blank" style="color: white; text-decoration:
none;">UserBenchmark.com</a>'

            st.markdown(f"""
<div class="ub-badge">
    <div class="ub-row">
        {gif_img}
        {logo_img}
    </div>
    <div class="ub-stat">{stat_line}</div>
</div>
""", unsafe_allow_html=True)
        else:
            gif_img = f'' if gif_b64
            else ""
            logo_img = f'<span class="ub-site"
style="opacity:0.8;">UserBenchmark.com</span>'
            st.markdown(f"""
<div class="ub-badge">
    <div
        class="ub-row">{gif_img}
{logo_img}</div>
    <div class="ub-stat" style="color:#aaa;">No
matching entry found</div>
</div>
""", unsafe_allow_html=True)

        with info_col:
            st.markdown(f"""
<div class="winner-card">
    <div class="gpu-name">{best_gpu['Name']}</div>
    <div class="mfg-badge">{mfg}</div>
    <div class="spec-grid">

```



```

        if pd.notna(img_url) and str(img_url).strip():
            gpu_img_html = f''
        else:
            gpu_img_html = '<div class="alt-gpu-img-
placeholder"></div>'

            if alt_ub_url and logo_b64:
                ub_logo_html = f'<a href="{alt_ub_url}"
target="_blank" style="margin-left: 6px;"></a>'
            elif alt_ub_url:
                ub_logo_html = f'<a href="{alt_ub_url}"
target="_blank" style="margin-left: 6px; color:#fff; font-
size:0.72rem; text-decoration:none;">[UB]</a>'
            else:
                ub_logo_html = ''

            st.markdown(f"""
<div class="alt-card">
    <div class="alt-
rank">#{int(row['Ranking'])}</div>
    {gpu_img_html}
    <div style="min-width:46px;">
        <span style="background:{alt_color};
color:#fff; font-size:0.6rem; font-weight:700; letter-
spacing:0.1em; padding:2px 7px; border-radius:3px; text-
transform:uppercase; box-shadow: none !important; text-shadow:
none !important;">{alt_mfg}</span>
    </div>
    <div style="flex:1; min-width:0;">
        <div class="alt-
name">{row['Name']}{ub_logo_html}</div>
        <div class="alt-meta">{row['Memory']}
&nbsp;&bull;&nbsp;&nbsp;&nbsp; {int(row['Release_Year'])}</div>
    </div>
    <div style="text-align:right; flex-shrink:0;">
        <div class="alt-score">V: {row['Nilai
V']:.4f}</div>
        <div class="alt-price">Rp {int(row['Average
Price (IDR)']):,}</div>
        <div class="alt-bench">Score:
{row['Benchmark']}</div>
    </div>
    </div>
    """, unsafe_allow_html=True)

```

2. base.css

```

/* Global Tags (NVIDIA, AMD, Intel) */
span[data-baseweb="tag"]:has(span[title="NVIDIA"]) {
    background-color: #76B900 !important;
}

span[data-baseweb="tag"]:has(span[title="AMD"]) {
    background-color: #ED1C24 !important;
}

```

```

span[data-baseweb="tag"]:has(span[title="Intel"]) {
  background-color: #0071C5 !important;
}

span[data-baseweb="tag"]:has(span[title="NVIDIA"]) *,
span[data-baseweb="tag"]:has(span[title="AMD"]) *,
span[data-baseweb="tag"]:has(span[title="Intel"]) * {
  color: white !important;
}

/* Page 1 */
@property --num {
  syntax: '<integer>';
  initial-value: 0;
  inherits: false;
}

.hero-container {
  display: flex;
  flex-direction: row;
  justify-content: center;
  gap: 28px;
  padding: 3.5rem 0 2.5rem 0;
  font-family: -apple-system, BlinkMacSystemFont, "Inter",
"Segoe UI", Roboto, sans-serif;
  color: #FFFFFF;
  opacity: 0;
  animation: fadeIn 1s forwards 0.2s;
}

.hero-col-left {
  display: flex;
  flex-direction: column;
  align-items: flex-end;
}

.hero-col-right {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
}

.top-left,
.top-right {
  height: 3.8rem;
}

.bottom-left,
.bottom-right {
  height: 2.6rem;
  margin-top: 5px;
}

.top-left {
  display: flex;
  align-items: flex-end;
  padding-bottom: 7px;
}

```

```

.top-right {
  display: flex;
  align-items: baseline;
  gap: 12px;
}

.bottom-right {
  display: flex;
  align-items: center;
}

.hero-app-name {
  font-size: 0.70rem;
  font-weight: 600;
  text-transform: uppercase;
  letter-spacing: 4px;
  opacity: 0.5;
}

.hero-number {
  font-size: 3.8rem;
  font-weight: 800;
  line-height: 1;
  counter-reset: gpu-counter var(--num);
  animation: countUpAnim 2s ease-out forwards 0.4s;
}

.hero-number::after {
  content: counter(gpu-counter);
}

.hero-text-large {
  font-size: 2.2rem;
  font-weight: 300;
  letter-spacing: 0.5px;
  opacity: 0.9;
  line-height: 1;
  white-space: nowrap;
}

.brands-reveal {
  height: 2.6rem;
  overflow: hidden;
  position: relative;
}

.brands-rotor {
  display: flex;
  flex-direction: column;
  font-size: 2.4rem;
  font-weight: 700;
  letter-spacing: -0.5px;
  text-align: right;
  animation: spinBrands 6s infinite cubic-bezier(0.64, 0.04,
0.35, 1) 2.6s;
}

.brand-word {

```

```

    height: 2.6rem;
    line-height: 2.6rem;
  }

  @keyframes fadeIn {
    to {
      opacity: 1;
    }
  }

  @keyframes spinBrands {

    0%,
    15% {
      transform: translateY(0);
    }

    30%,
    45% {
      transform: translateY(-2.6rem);
    }

    60%,
    75% {
      transform: translateY(-5.2rem);
    }

    90%,
    100% {
      transform: translateY(-7.8rem);
    }
  }

  .viz-dashboard-header {
    display: flex;
    align-items: center;
    gap: 14px;
    padding: 1.2rem 1.4rem;
    margin: 2rem 0 1rem 0;
    border: 1px solid #222;
    border-radius: 12px;
    backdrop-filter: blur(8px);
    -webkit-backdrop-filter: blur(8px);
  }

  .viz-header-icon {
    font-size: 1.8rem;
    line-height: 1;
  }

  .viz-header-title {
    font-size: 1.15rem;
    font-weight: 700;
    color: #e0e0e0;
    letter-spacing: 0.02em;
  }

  .viz-header-sub {
    font-size: 0.78rem;

```

```

        color: #777;
        margin-top: 2px;
    }

    .viz-control-bar {
        padding: 0.2rem 0;
        margin-bottom: 0.6rem;
    }

    [data-testid="stPlotlyChart"] {
        border-radius: 8px;
        overflow: hidden;
    }

    [data-testid="stContainer"] {
        transition: border-color 0.2s ease;
    }

    [data-testid="stContainer"]:hover {
        border-color: #383838 !important;
    }

    /* Page 3 */
    .p3-root {
        font-family: "Source Sans Pro", sans-serif;
    }

    .p3-header {
        font-size: 1.75rem;
        font-weight: 700;
        color: #fafafa;
        margin-bottom: 0.15rem;
    }

    .p3-sub {
        font-size: 0.82rem;
        color: #888;
        margin-bottom: 1.4rem;
    }

    /* Winner card */
    .winner-card {
        background: #161616;
        border: 1px solid #272727;
        border-left: 4px solid var(--mfg-color);
        border-radius: 10px;
        padding: 1.5rem 1.8rem;
        margin-bottom: 0.8rem;
    }

    .gpu-name {
        font-size: 1.6rem;
        font-weight: 700;
        color: #fff;
        margin-bottom: 0.25rem;
    }

    .mfg-badge {

```

```

display: inline-block;
background: var(--mfg-color);
color: #fff;
font-size: 0.7rem;
font-weight: 700;
letter-spacing: 0.12em;
padding: 2px 10px;
border-radius: 3px;
margin-bottom: 1.1rem;
text-transform: uppercase;
box-shadow: none !important;
text-shadow: none !important;
}

.spec-grid {
display: grid;
grid-template-columns: repeat(auto-fill, minmax(185px, 1fr));
gap: 0.6rem;
}

.spec-item {
background: #1d1d1d;
border: 1px solid #2a2a2a;
border-radius: 7px;
padding: 0.6rem 0.85rem;
}

.spec-label {
font-size: 0.65rem;
text-transform: uppercase;
letter-spacing: 0.1em;
color: #666;
margin-bottom: 0.15rem;
}

.spec-value {
font-size: 0.95rem;
font-weight: 600;
color: #e0e0e0;
}

/* SAW score */
.score-box {
background: var(--mfg-bg);
border: 1px solid var(--mfg-border);
border-radius: 9px;
padding: 0.9rem 1rem;
text-align: center;
margin-top: 0.75rem;
}

.score-lbl {
font-size: 0.68rem;
text-transform: uppercase;
letter-spacing: 0.1em;
color: #777;
margin-bottom: 0.2rem;
}

```

```

.score-val {
    font-size: 2.2rem;
    font-weight: 700;
    color: var(--mfg-color);
    line-height: 1;
}

/* Userbenchmark badge */
.ub-badge {
    background: #111;
    border: 1px solid #2a2a2a;
    border-radius: 9px;
    padding: 0.85rem 1rem;
    margin-top: 0.75rem;
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 0.3rem;
}

.ub-row {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}

.ub-site {
    font-size: 1.05rem;
    font-weight: 700;
    color: #fff;
    text-decoration: none;
}

.ub-site:hover {
    text-decoration: underline;
}

.ub-stat {
    font-size: 0.78rem;
    color: #888;
}

.ub-stat strong {
    color: #bbb;
}

/* Alt section */
.alt-section-title {
    font-size: 0.75rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.12em;
    color: #888;
    margin: 1.6rem 0 0.7rem 0;
    padding-bottom: 0.4rem;
    border-bottom: 1px solid #222;
}

.alt-card {

```

```

        background: #141414;
        border: 1px solid #222;
        border-radius: 9px;
        padding: 0.75rem 1rem;
        display: flex;
        align-items: center;
        gap: 0.9rem;
        margin-bottom: 0.55rem;
    }

    .alt-card:hover {
        border-color: #383838;
    }

    .alt-rank {
        font-size: 1.3rem;
        font-weight: 700;
        color: #3a3a3a;
        min-width: 30px;
        text-align: center;
    }

    .alt-gpu-img {
        width: 72px;
        height: 46px;
        object-fit: contain;
        border-radius: 4px;
        background: #1a1a1a;
        flex-shrink: 0;
    }

    .alt-gpu-img-placeholder {
        width: 72px;
        height: 46px;
        background: #1e1e1e;
        border-radius: 4px;
        flex-shrink: 0;
    }

    .alt-name {
        font-size: 0.92rem;
        font-weight: 600;
        color: #ddd;
    }

    .alt-meta {
        font-size: 0.72rem;
        color: #666;
        margin-top: 2px;
    }

    .alt-ub-gif {
        width: 36px;
        height: 36px;
        object-fit: contain;
        flex-shrink: 0;
        cursor: pointer;
        opacity: 0.85;
        transition: opacity 0.2s;
    }

```

```

}

.alt-ub-gif:hover {
    opacity: 1;
}

.alt-score {
    font-size: 0.9rem;
    font-weight: 700;
    color: #aaa;
    white-space: nowrap;
}

.alt-price {
    font-size: 0.75rem;
    color: #999;
    white-space: nowrap;
}

.alt-bench {
    font-size: 0.72rem;
    color: #666;
}

```

3. DataProcessor.ipynb

File ini merupakan Jupyter Notebook berbasis Python yang difokuskan khusus pada tahap persiapan dan pembersihan data (Data Preparation & Preprocessing). Fungsi utama dari script ini adalah untuk membaca dataset mentah yang berasal dari berbagai sumber (seperti spesifikasi GPU dari TechPowerUp, data harga pasar dari hasil web scraping, dan skor performa dari UserBenchmark). Di dalam file ini dilakukan proses penggabungan data (merging), penanganan data yang kosong (missing values), serta penyesuaian format tipe data (misalnya mengonversi format harga teks menjadi numerik murni). Hasil akhir atau keluaran (output) dari script ini adalah satu file utama bernama dataset.csv yang sudah bersih, terstruktur, dan siap diproses ke tahap perhitungan SPK.

```

import pandas as pd

tpu_data = pd.read_csv("Data/Used Data/techpowerup_gpus_with_images.csv")
price_data = pd.read_csv("Data/Used Data/all_scraped_gpu_prices.csv")
ub_data = pd.read_csv("Data/Used Data/GPU_UserBenchmarks.csv")

def format_ub_name(row):
    brand = str(row['Brand'])
    model = str(row['Model'])

    model = model.replace('-S (Super)', 'SUPER')
    model = model.replace('-TS (Ti-Super)', 'Ti SUPER')
    model = model.replace('-', ' ')

    if brand == 'Nvidia': return 'GeForce ' + model
    elif brand == 'AMD': return 'Radeon ' + model

```

```

elif brand == 'Intel': return model
return model

ub_data['Mapped_Name'] = ub_data.apply(format_ub_name, axis=1)

ub_avg =
ub_data.groupby('Mapped_Name')[['Benchmark']].mean().reset_index(
)

ub_mapped_names = sorted(ub_avg['Mapped_Name'].unique(), key=len,
reverse=True)

def find_best_ub_match(tpu_name):
    if tpu_name in ub_mapped_names:
        return tpu_name

    for ub_name in ub_mapped_names:
        if tpu_name.startswith(ub_name + " "):
            return ub_name

    return None

tpu_data['UB_Match_Name'] =
tpu_data['Name'].apply(find_best_ub_match)

category_prices = price_data[['GPU_Category',
'Average_Category_Price_IDR']].drop_duplicates()

dataset = pd.merge(
    tpu_data,
    category_prices,
    left_on='Name',
    right_on='GPU_Category',
    how='inner'
)

dataset = pd.merge(
    dataset,
    ub_avg,
    left_on='UB_Match_Name',
    right_on='Mapped_Name',
    how='left'
)

dataset['Benchmark'] = dataset['Benchmark'].fillna(0)

dataset = dataset.drop(columns=['GPU_Category', 'Mapped_Name',
'UB_Match_Name']).rename(
    columns={'Average_Category_Price_IDR': 'Average Price (IDR)'}
)

dataset.to_csv("Data/dataset.csv", index=False)

print(f"Total GPUs matched and saved: {len(dataset)}")
display(dataset.head())

```

4. scrapperEnterKomputer.ipynb

```
from bs4 import BeautifulSoup
import pandas as pd
import re
from IPython.display import display

# load TechPowerUp GPU csv
gpu_df = pd.read_csv("techpowerup_gpus_with_images.csv")

# Advanced Normalization
def normalize_gpu_name(name):
    n = str(name).lower()
    for vendor in ['geforce ', 'radeon ', 'intel ', 'arc ']:
        n = n.replace(vendor, '')
    n = re.sub(r'\s+gb', 'gb', n)
    return n.strip()

# Prepare Search Keys and Sort by Length (Longest first to prevent
overlaps like '5060' matching '5060 ti')
gpu_df['Search_Key'] = gpu_df['Name'].apply(normalize_gpu_name)
search_keys_df = gpu_df[['Name', 'Search_Key']].copy()
search_keys_df['Key_Len'] = search_keys_df['Search_Key'].apply(len)
search_keys_df = search_keys_df.sort_values(by='Key_Len',
ascending=False)

# Read the EnterKomputer HTML file
file_path = "Category_VGA - Enterkomputer Toko Online Komputer,
Simulasi & Rakit PC.html"
with open(file_path, "r", encoding="utf-8") as f:
    html_content = f.read()

soup = BeautifulSoup(html_content, 'html.parser')

# Extract Product Titles and Safe Prices from HTML
scraped_items = []
for prod in soup.find_all('a', class_='ps-product__title'):
    title = prod.text.strip()

    parent_div = prod.find_parent('div', class_='ps-product__content')
    if not parent_div: continue

    price_p = parent_div.find('p', class_='ps-product__price')
    if not price_p: continue

    full_price_text = price_p.get_text(separator=" ", strip=True)

    matches = re.findall(r'Rp\s?([\d\.]+)', full_price_text)
    if matches:
        price_str = matches[-1].replace('.', '')
        if price_str.isdigit():
            scraped_items.append({
                'Store_Title': title,
                'Normalized_Title': title.lower().replace(' gb',
'gb'),
                'Price_IDR': int(price_str)
            })
```

```

matched_data = []

for item in scraped_items:
    store_title_lower = item['Normalized_Title']
    matched_target_name = None

    for _, row in search_keys_df.iterrows():
        key_parts = row['Search_Key'].split()
        if all(part in store_title_lower for part in key_parts):
            matched_target_name = row['Name']
            break

    if matched_target_name:
        matched_data.append({
            'GPU_Category': matched_target_name,
            'Vendor/Model_Title': item['Store_Title'],
            'Model_Price_IDR': item['Price_IDR']
        })

df_detailed = pd.DataFrame(matched_data)

if not df_detailed.empty:

    avg_prices = df_detailed.groupby('GPU_Category')['Model_Price_IDR'].mean().round().astype(int).reset_index()
    avg_prices.rename(columns={'Model_Price_IDR': 'Average_Category_Price_IDR'}, inplace=True)

    # Merge detailed view with averages
    df_presentation = pd.merge(df_detailed, avg_prices, on='GPU_Category', how='left')

    # Sort
    df_presentation = df_presentation.sort_values(by=['GPU_Category', 'Model_Price_IDR'], ascending=[True, True])

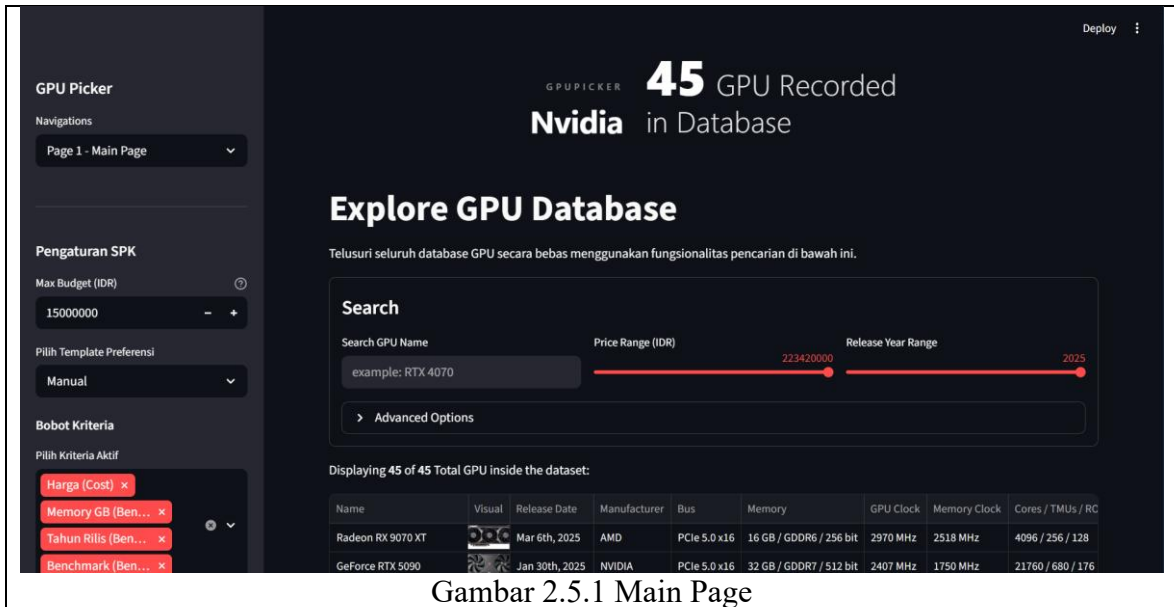
    print(f"\n--- DETAILED SCRAPED DATA ({len(df_presentation)} rows) ---")
    display(df_presentation)

    # save data to csv
    out_file = "all_scraped_gpu_prices.csv"
    df_presentation.to_csv(out_file, index=False)
    print(f"\nSaved all {len(df_presentation)} scraped rows detailed view to '{out_file}'!")
else:
    print("Warning: No matching prices could be found.")

```

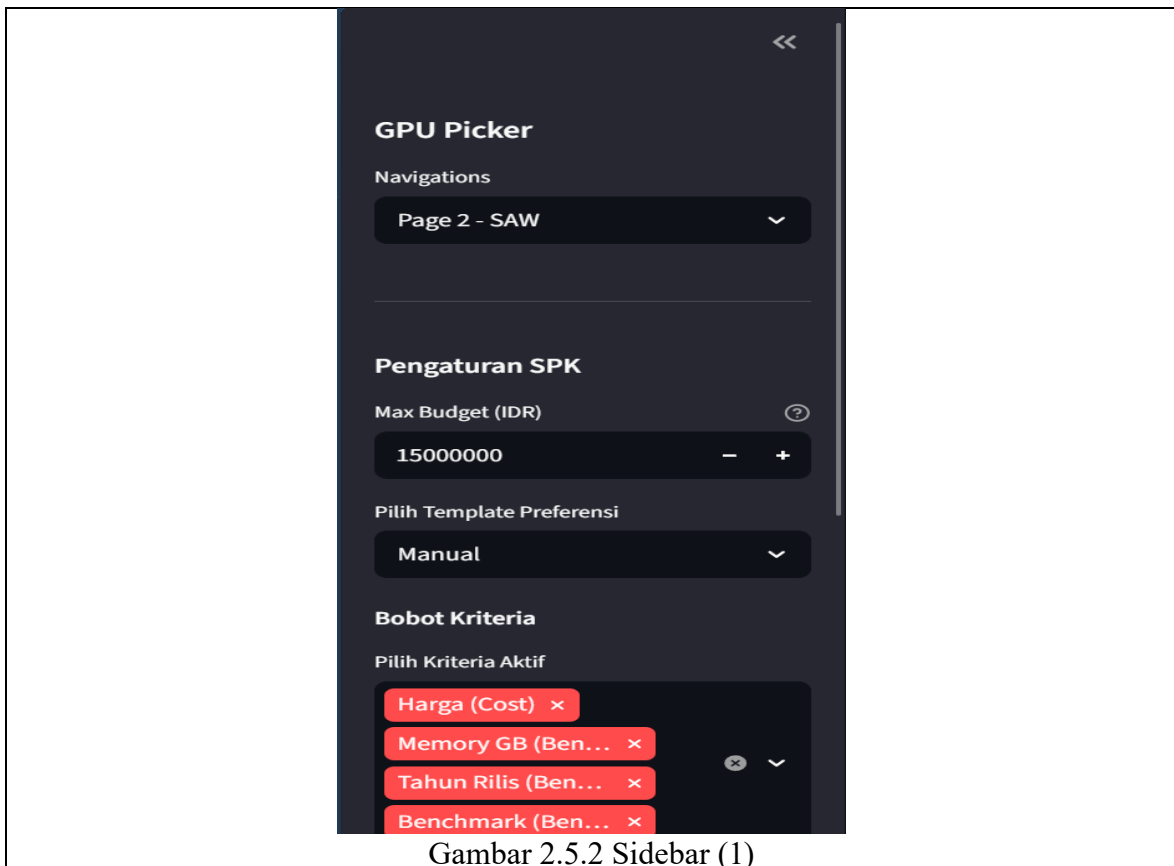
B. Screenshot Sistem

1. Page 1 - Main Page

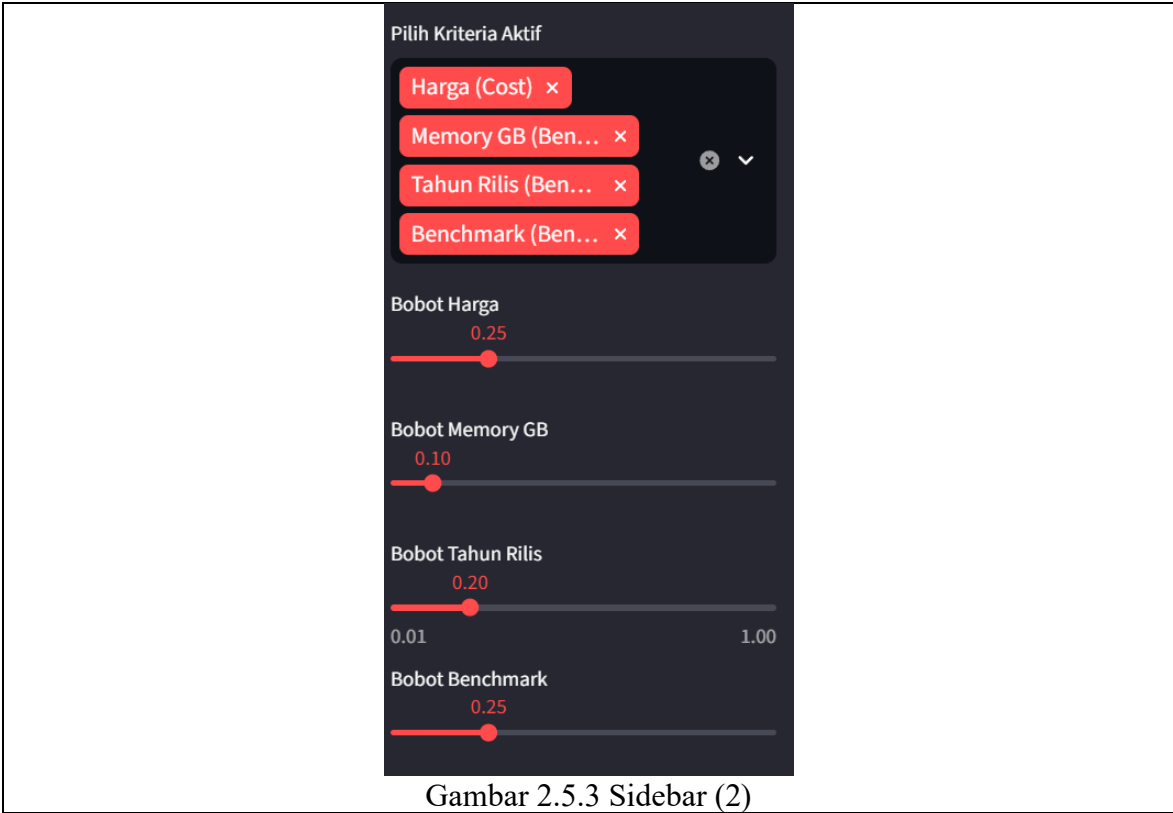


Gambar 2.5.1 Main Page

2. Sidebar



Gambar 2.5.2 Sidebar (1)



3. Page 2 - SAW

GPU Picker

Page 2 - SAW

Pengaturan SPK

Max Budget (IDR)

15000000

Pilih Template Preferensi

Manual

Bobot Kriteria

Pilih Kriteria Aktif

Harga (Cost) x

Memory GB (Ben... x

Tahun Rilis (Ben... x

Benchmark (Ben... x

Deploy

Metode Simple Additive Weighting (SAW)

Menampilkan matriks normalisasi SAW hanya untuk GPU yang lolos validasi budget.

1. Inisialisasi Matriks Awal (x)

	Price (IDR)	Memory (GB)	Tahun Rilis	Benchmark Score
Arc B580	5575000	12	2024	46.1
GeForce GTX 1050 Ti	2226667	4	2016	12.2
GeForce GTX 1650	2047000	4	2019	18.1
Radeon RX 7800 XT	9030000	16	2023	75.6
GeForce RTX 3050 8 GB	4705833	8	2022	31
GeForce RTX 4070 SUPER	10336667	12	2024	80
GeForce RTX 5060 Ti 8 GB	7944400	8	2025	71.4
GeForce RTX 4070 Ti SUPER	11500000	16	2024	101
GeForce RTX 4060 Ti 8 GB	6509818	8	2023	60.9

Gambar 2.5.4 SAW (1)

2. Matriks Keputusan Berdasarkan Atribut (k)

Keterangan: Harga di-set 0 (Cost), Selain itu (Tahun Rilis, Core, Benchmark) di-set 1 (Benefit)

	Price (IDR)	Memory (GB)	Tahun Rilis	Benchmark Score
Atribut (0=Biaya, 1=Keuntungan)	0	1	1	1

3. Normalisasi Matriks (R)

$$r_{ij} = \begin{cases} \frac{x_{ij}}{\max_i x_{ij}} & \text{jika } j \text{ adalah atribut benefit (keuntungan)} \\ \frac{\min_i x_{ij}}{x_{ij}} & \text{jika } j \text{ adalah atribut cost (biaya)} \end{cases}$$

	Price (IDR)	Memory (GB)	Tahun Rilis	Benchmark Score
GeForce RTX 4060 Ti 8 GB	0.1591	0.5000	0.9990	0.6030
GeForce RTX 4070 Ti	0.0857	0.7500	0.9990	1.0000
Radeon RX 6600 XT	0.2089	0.5000	0.9980	0.4485
GeForce GTX 1660 Ti	0.3378	0.3750	0.9970	0.3149
GeForce GTX 750 Ti	0.8780	0.1250	0.9946	0.0712
Radeon RX 7700 XT	0.1385	0.7500	0.9990	0.6446

Gambar 2.5.5 SAW (2)

4. Bobot (w)

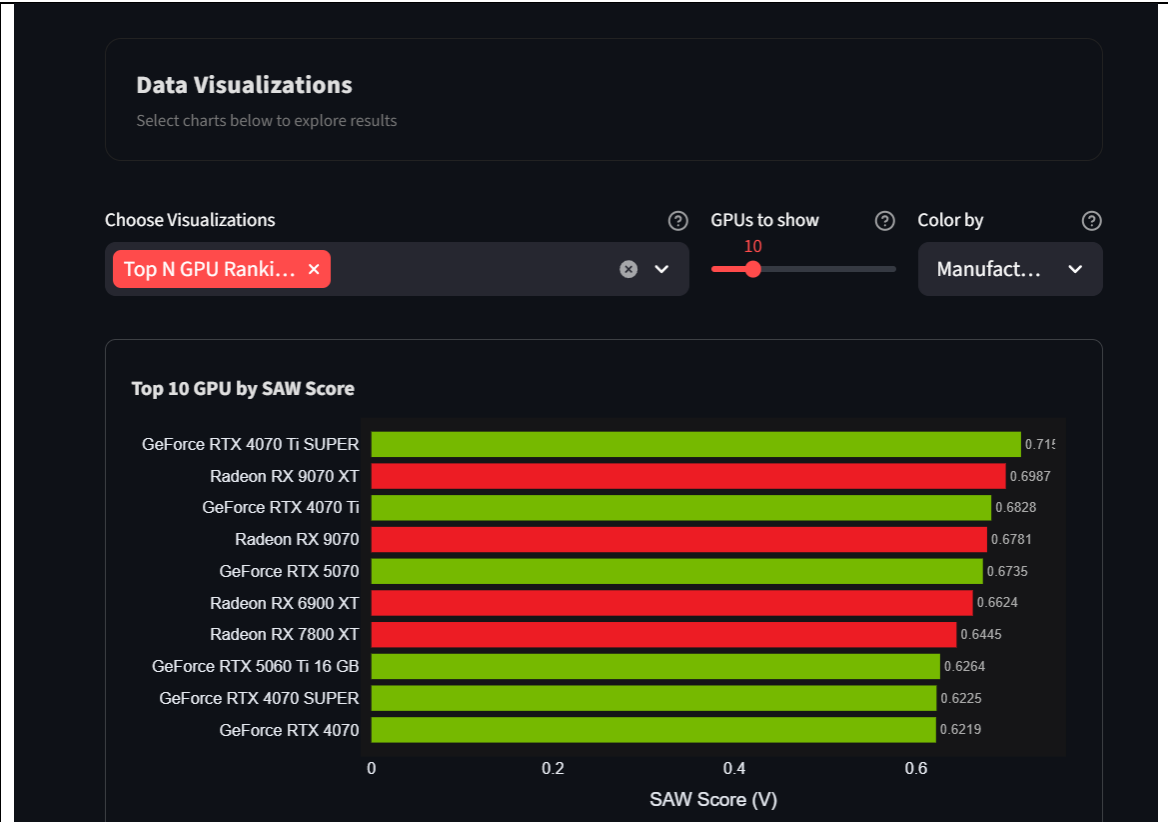
	Price (IDR)	Memory (GB)	Tahun Rilis	Benchmark Score
Bobot W (Ternormalisasi)	0.3125	0.1250	0.2500	0.3125

5. Hasil Perangkingan Akhir (V)

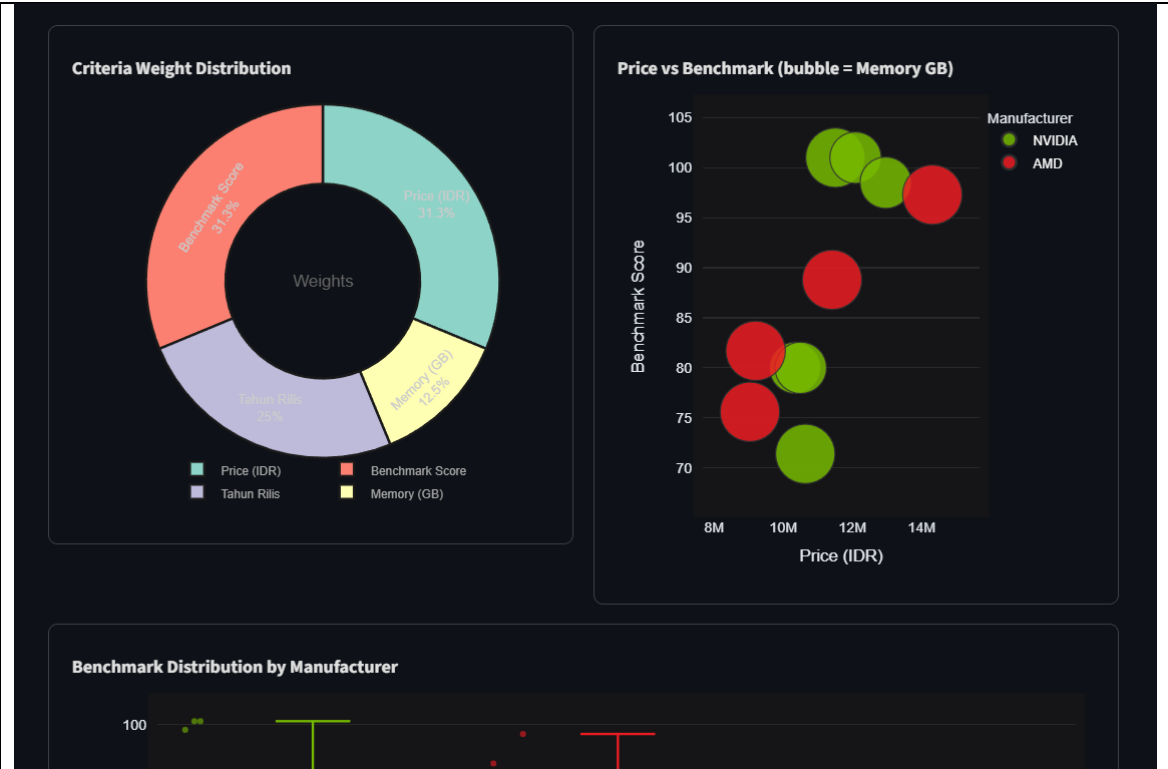
$$V_i = \sum_{j=1}^n w_j r_{ij}$$

	Nilai V
GeForce RTX 4070	0.62100
Radeon RX 9060 XT 16 GB	0.61033
Radeon RX 550	0.59555
Radeon RX 7700 XT	0.58821
GeForce RTX 5060 Ti 8 GB	0.57417
Radeon RX 9060 XT 8 GB	0.56211
GeForce GTX 750 Ti	0.56088

Gambar 2.5.6 SAW (3)



Gambar 2.5.6 SAW (4)

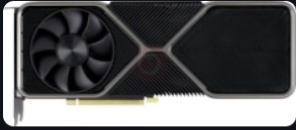


Gambar 2.5.7 SAW (5)

4. Page 3 - Result & Recommendation

GPU Recommendation

GPU yang paling sesuai berdasarkan Max Budget



SAW SCORE (NILAI V)

0.7155



UserBenchmark.com

GPU Rank #8 • Average
Benchmark: 105.0%

GeForce RTX 4070 Ti SUPER

NVIDIA

PRICE	BENCHMARK SCORE
Rp 11,500,000	101.0
RELEASE DATE	MEMORY
Jan 8th, 2024	16 GB / GDDR6X / 256 bit
GPU CLOCK	CORES / TMUS / ROPS
2610 MHz	8448 / 264 / 96
BUS INTERFACE	
PCIe 4.0 x16	

Gambar 2.5.8 Result & Recommendation (1)

ALTERNATIVE OPTIONS — RANK 2 TO 5

#2		AMD	Radeon RX 9070 XT 	V: 0.6987 Rp 14,302,368 Score: 97.3
#3		NVIDIA	GeForce RTX 4070 Ti 	V: 0.6828 Rp 12,083,333 Score: 101.0
#4		AMD	Radeon RX 9070 	V: 0.6781 Rp 11,405,000 Score: 88.8
#5		NVIDIA	GeForce RTX 5070 	V: 0.6735 Rp 12,956,452 Score: 98.5

Gambar 2.5.9 Result & Recommendation (2)

5. DataProcessor.ipynb

Total GPUs matched and saved: 45

	Name	Image_URL	Release Date	Bus	Memory	GPU Clock	Memory Clock	Cores / TMUs / ROPs	Avera Pri (ID)
0	Radeon RX 9070 XT	https://tpucdn.com/gpu-specs/images-new/c/4229...	Mar 6th, 2025	PCIe 5.0 x16	16 GB / GDDR6 / 256 bit	2970 MHz	2518 MHz	4096 / 256 / 128	143023
1	GeForce RTX 5090	https://tpucdn.com/gpu-specs/images-new/c/4216...	Jan 30th, 2025	PCIe 5.0 x16	32 GB / GDDR7 / 512 bit	2407 MHz	1750 MHz	21760 / 680 / 176	613349
2	GeForce RTX 5070	https://tpucdn.com/gpu-specs/images-new/c/4218...	Mar 4th, 2025	PCIe 5.0 x16	12 GB / GDDR7 / 192 bit	2512 MHz	1750 MHz	6144 / 192 / 80	129564
3	Radeon RX 9060 XT 16 GB	https://tpucdn.com/gpu-specs/images-new/c/4293...	Jun 4th, 2025	PCIe 5.0 x16	16 GB / GDDR6 / 128 bit	3130 MHz	2518 MHz	2048 / 128 / 64	88225
4	GeForce RTX 5070 Ti	https://tpucdn.com/gpu-specs/images-new/c/4243...	Feb 20th, 2025	PCIe 5.0 x16	16 GB / GDDR7 / 256 bit	2452 MHz	1750 MHz	8960 / 280 / 96	172243

Gambar 2.5.10 Hasil DataProcessor

6. scraperpowerup.ipynb

	Name	Image_URL	Release Date	Bus	Memory	GPU Clock	Memory Clock	Cores / TMUs / ROPs
0	Radeon RX 9070 XT		Mar 6th, 2025	PCIe 5.0 x16	16 GB / GDDR6 / 256 bit	2970 MHz	2518 MHz	4096 / 256 / 128
1	GeForce RTX 5090		Jan 30th, 2025	PCIe 5.0 x16	32 GB / GDDR7 / 512 bit	2407 MHz	1750 MHz	21760 / 680 / 176
2	GeForce RTX 5070		Mar 4th, 2025	PCIe 5.0 x16	12 GB / GDDR7 / 192 bit	2512 MHz	1750 MHz	6144 / 192 / 80

Gambar 2.5.11 Hasil scraperpowerup

7. scraperEnterKomputer.ipynb

--- DETAILED SCRAPED DATA (443 rows) ---

	GPU_Category	Vendor/Model_Title	Model_Price_IDR	Average_Category_Price_IDR
277	Arc B580	MAXSUN Intel ARC B580 12GB GDDR6 Milestone 12G...	4950000	5575000
410	Arc B580	SPARKLE Intel ARC B580 12GB GDDR6 Titan OC - B...	4950000	5575000
15	Arc B580	MAXSUN Intel ARC B580 12GB GDDR6 iCraft 12G (W...	5850000	5575000
276	Arc B580	MAXSUN Intel ARC B580 12GB GDDR6 iCraft 12G (W...	5850000	5575000
62	Arc B580	ASRock Intel ARC B580 12GB GDDR6 - Challenger ...	5875000	5575000
...	...	...	...	...
115	Radeon RX 9070 XT	Asus Radeon RX 9070 XT 16GB GDDR6 TUF Gaming O...	14950000	14302368
385	Radeon RX 9070 XT	Sapphire NITRO+ Radeon RX 9070 XT 16GB GDDR6 G...	14950000	14302368
231	Radeon RX 9070 XT	Gigabyte Radeon RX 9070 XT 16GB GDDR6 Aorus El...	15050000	14302368
378	Radeon RX 9070 XT	PowerColor Radeon RX 9070 XT 16GB GDDR6 Hellho...	15395000	14302368
63	Radeon RX 9070 XT	ASRock Radeon RX 9070 XT 16GB GDDR6 Taichi Whi...	15970000	14302368

Gambar 2.5.12 Hasil scraperEnterKomputer

BAB III

JADWAL Pengerjaan DAN PEMBAGIAN TUGAS

3.1 Jadwal Pengerjaan

No	Kegiatan	Mei 2026			
		Minggu			
		1	2	3	4
1	Mengkaji Masalah				
2	Perancangan Program				
3	Pengumpulan Data				
4	Pembuatan Program				
5	Laporan				

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Dimas Hafid Fathoni	123240159	- Mengkaji Masalah - Pengumpulan Data - Pembuatan Program - Laporan
2	Jauza Ilha Mahardhika Putra	123240174	- Perancangan Progran - Pengumpulan Data - Pembuatan Program - Laporan

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat ditarik kesimpulan bahwa proyek ini telah berhasil membangun sebuah Sistem Pendukung Keputusan (SPK) berbasis web menggunakan *framework Streamlit* untuk mempermudah pengguna dalam memilih *Graphics Processing Unit (GPU)* yang paling optimal. Sistem ini berhasil menerapkan algoritma *Simple Additive Weighting (SAW)* untuk merangking alternatif *GPU* secara objektif berdasarkan matriks spesifikasi (seperti memori, *clock*, jumlah *core*, tahun rilis) dan performa benchmark. Selain itu, SPK ini terbukti fleksibel karena mengintegrasikan berbagai sumber data secara terpadu dan memungkinkan pengguna untuk menyesuaikan bobot kriteria sesuai dengan batasan anggaran serta preferensi kebutuhan mereka (seperti *Gaming*, *Price to Performance*, maupun *Content Creation*).

4.3 Saran

Untuk penyempurnaan dan pengembangan sistem lebih lanjut di masa mendatang, terdapat beberapa saran yang dapat dipertimbangkan:

- **Pembaruan Data Secara Otomatis**

Mengembangkan sistem web scraping agar dapat memperbarui data harga dan ketersediaan stok *GPU* di pasaran secara real-time atau otomatis terjadwal, sehingga rekomendasi harga selalu akurat dan relevan dengan fluktuasi pasar.

- **Penambahan Kriteria Spesifikasi Teknis**

Menambahkan kriteria evaluasi lain yang sering menjadi pertimbangan dalam merakit PC, seperti konsumsi daya maksimal (*TDP*), dimensi ukuran (panjang *GPU*), atau teknologi fitur tambahan (seperti dukungan *Ray Tracing* atau *Upscaling AI*).

- **Perluasan Rekomendasi Komponen**

Mengembangkan cakupan sistem agar tidak hanya merekomendasikan *GPU*, tetapi juga memberikan saran atau peringatan terkait potensi bottleneck apabila *GPU* yang dipilih dipasangkan dengan prosesor (*CPU*) tertentu.

DAFTAR PUSTAKA

- Enterkomputer. (2026). *Katalog Produk dan Harga VGA*. Diakses pada tahun 2026, dari <https://www.enterkomputer.com/>
- Prodi Informatika UPN "Veteran" Yogyakarta. (2026). *Modul Praktikum Kecerdasan Buatan*. Yogyakarta: Universitas Pembangunan Nasional "Veteran" Yogyakarta.
- TechPowerUp. (2026). *TechPowerUp GPU Specs Database*. Diakses pada tahun 2026, dari <https://www.techpowerup.com/gpu-specs/?mfgr=>
- UserBenchmark. (2026). *UserBenchmark Official Developer CSV Data*. Diakses pada tahun 2026, dari <https://www.userbenchmark.com/page/developer>
- Putra, J. I. M. dan Fathoni, D. H. (2026). *dragonsterm/gpu-picker-saw*. Diakses pada tahun 2026, dari <https://github.com/dragonsterm/gpu-picker-saw>